

编译原理课程项目验收

Pascal-S 编译器

从词法分析到语义检查与 C 代码生成的完整流水线

源码入口: `src/main.cpp` · 幻灯片: `docs/slides.typ`

Lexer

LL(1)

AST

Semantic

C Codegen

验收讲解路线

1. 结构与流程

项目目录、主流程、输入输出文件、演示命令。

2. 符号表

表项结构、作用域栈、查找、插入、定位与重定位。

3. 词法分析

token 翻译表、注释处理、错误检测、与 Parser 接口。

4. 语法分析

LL(1) 实现、FIRST/FOLLOW、预测表、AST 构建、语法错误。

5. 语义分析

支持类型、表达式/数组/参数/调用/语句检查。

6. 代码生成与测试

C 代码生成策略、错误恢复、测试用例编译过程与结果。

项目总体结构

目录组织

- src/lexer: Token、扫描器、词法错误
- src/parser: 文法、LL(1) 预测分析、AST
- src/semantic: 符号表与语义检查
- src/codegen: Pascal-S 到 C 的生成器
- tests: 正确样例、词法错误、语法错误、语义错误、集成测试

主程序入口

src/main.cpp 负责命令行解析，并按阶段调度：

- 构造文法并转换为 LL(1)
- 词法 + 语法分析并构建 AST
- 语义分析通过后生成 C
- 输出 tokens.txt、parse_process.txt、ast.txt、目标 .c

完整编译流水线

1 源程序

.pas 文件; 也保留旧 token 文件兼容入口。

2 词法

Lexer::nextToken()
跳过空白/注释并返回带行列号的 token。

3 语法

Parser::parse()
根据预测分析表进行 PREDICT / MATCH / REDUCE。

4 语义

SemanticAnalyzer::
遍历 AST, 构建符号表并检查类型。

5 生成

CCodeGenerator::generate()
输出可编译 C 源码。

运行示例

Pascal9 编译器验收显示
`src/pascal_compiler.exe -i tests/correct_test/correct_all.pas -o /private/tmp/correct_all_demo.c` 2026 · Compiler Project

讲解分工建议

成员 A: 词法与接口

- 讲 token.h 的 TokenType / 翻译表
- 讲 lexer.cpp 的注释跳过、数字/字符串/标识符扫描
- 演示词法错误与 token 输出

成员 B: 语法与 AST

- 讲 grammar.cpp 的文法改写、FIRST/FOLLOW、预测表
- 讲 parser.cpp 的分析栈、语义栈、规约动作
- 演示 parse_process.txt 与 ast.txt

成员 C: 符号表与语义

Pascal S 编译器验收展示

- 讲 symbol_table.* 的作用域栈与表项结构
- 讲 semantic_analyzer.cpp 的类型推断和错误

成员 D: 代码生成与测试

- 讲 codegen.cpp 的 Visitor 输出策略
- 讲 Pascal 类型、数组、record、调用到 C 的

2026 Compiler Project

02 SYMBOL TABLE

符号表

内容、结构，以及查找 / 插入 / 定位 / 重定位操作

Pascal-S 编译器验收展示

符号表内容与表项结构

TypeInfo: 类型描述

- Category: Simple、Array、Record
- 简单类型: integer、real、boolean、char、string
- 数组: arrayLow、arrayHigh、elementType
- 记录: recordFields = [(fieldName, TypeInfo)]
- 提供 fromString() / toString() / findField()

Symbol: 符号表项

- name: 标识符名, 词法层已统一为小写
- kind: 常量、变量、参数、函数、过程、程序
- type: 完整 TypeInfo
- scopeLevel: 声明所在层级
- params / returnType: 函数过程签名
- constValue、byReference: 常量值与 var 参数

符号表的组织方式

作用域栈

SymbolTable 内部是:

```
vector<unordered_map<string, Symbol>>  
scopes_;
```

每个 unordered_map 表示一个作用域; 栈顶是当前作用域。

为什么这样设计

- 插入只发生在当前作用域
- 查找从栈顶向外层扫描, 天然支持遮蔽
- 进入函数/过程时 enterScope()
- 离开函数/过程时 exitScope()
- 当前层级由 getCurrentLevel() 统一计算

符号表操作：查找与插入

插入 declare

- 只向 `scopes_.back()` 插入
- 若当前作用域已有同名符号，返回 `false`
- 插入前补上 `scopeLevel`
- 用于检测重复常量、变量、参数、子程序声明

查找 lookup

- 从最内层作用域向外层作用域扫描
- 命中第一个符号即返回
- `lookupCurrent()` 只查当前层
- `lookupCallable()` 只接受 Function / Procedure
- `lookupFunction()` 用于函数表达式

符号表操作：定位与重定位

定位

- `enterScope()` 创建新的当前作用域
- `scopeLevel` 标记声明层级
- 使用 `lookup()` 将变量引用定位到最近的可见声明
- 函数内部把函数名声明为局部返回值变量
- `line/column` 让错误定位到 AST 节点来源

重定位

- `exitScope()` 弹出当前作用域
- 同名标识符自动恢复到外层绑定
- 这相当于离开作用域后的符号重定位
- 代码生成阶段进一步做地址级调整：数组下界偏移、`var` 参数取地址/解引用

03 LEXER

词法分析

翻译表、注释处理、错误检测，以及与语法分析器的接口

Pascal-S 编译器验收展示

词法翻译表

TokenType 到文法终结符

token.h 中的 `tokenTypeToTerminal()` 把枚举映射到 LL(1) 文法终结符:

- PROGRAM $\rightarrow$ program
- ASSIGN $\rightarrow$ assignop
- NUM $\rightarrow$ num
- STRING_KW $\rightarrow$ string_kw
- END_OF_FILE $\rightarrow$ \$

关键字表

`Lexer::keywordMap()` 统一维护关键字:

- 控制流: if、for、while、case
- 类型: integer、real、boolean、char、string
- 扩展: downto、break、continue、record
- 运算: div、mod、and、or、not

词法扫描流程

1. 预处理

`skipWhitespaceAndComments()`
跳过空白和注释。

2. 分类扫描

数字、标识符/关键字、字符/字符串、运算符和界符。

3. 产出 token

每个 token 都携带 `lexeme`、`line`、`column`。

注释处理

支持三类注释:

- `{ ... }`
- `(* ... *)`
- `// ...`

Pascal S 编译器验收展示

错误检测

- 标识符不能以数字开头
- 实数小数点后必须有数字
- 多个小数点非法
- 非 ASCII 字符非法

2026 · Compiler Project

词法与语法分析器接口

按需扫描

Parser 持有非拥有指针 `Lexer* lexer`, 构造时读取第一个 lookahead:

```
currentTok = lexer->nextToken();
```

匹配终结符成功后调用 `advance()`, 再取下一个 token。

接口收益

- 不需要先生成完整 token 序列
- Parser 可同步收集已消费 token
- 词法错误保留在 `lexer.getErrors()`
- verbose 模式写出 `tokens.txt`

语法分析

LL(1) 表驱动实现、AST 构建、可检测语法错误和输出

Pascal-S 编译器验收展示

语法分析实现方法

文法准备

Grammar 保存产生式:

```
struct Production { string lhs;  
vector<string> rhs; };
```

编译启动时执行:

- 提取左公因子
- 消除左递归
- 计算 FIRST / FOLLOW

• 构造 LR(0) 预测分析表

表驱动分析

Parser::parse() 使用分析栈:

- 栈顶是终结符: 尝试 MATCH
- 栈顶是非终结符: 查表 PREDICT
- 栈顶是动作标记: 执行 REDUCE
- 输入结束且栈顶 \$: ACCEPT

AST 构建机制

不是事后补树

AST 构建嵌入 LL(1) 分析:

- 终结符匹配后把 token 压入语义栈
- 产生式右部完成后触发动作标记
- 从语义栈取出右部值并规约
- 最终 programstruct 规约成 AST 根

节点覆盖

- 声明: const、var、parameter、subprogram
- 语句: assign、call、if、for、while、case、break、continue、read、write
- 表达式: literal、variable、unary、binary、call
- 复合结构: array index、record field、函数/过程实参

能检测哪些语法错误

程序结构错误

缺少 program、缺少 .、缺少 end、
多余 token。

语句结构错误

缺少 then、缺少 do、不完整赋值、
非法语句开头。

表达式/类型声明错误

表达式不完整、括号不闭合、数
组/类型声明不完整。

错误输出

每条错误包含:

- 阶段: Parser
- 源码位置: Line / Col
- 当前 token

Pascal-3 编译器验收展示

恢复策略

采用 panic mode:

- 若当前 token 在 FOLLOW 集中, 弹出当前非
终结符
- 否则跳过输入, 直到遇到 FIRST 或 FOLLOW

2026-1 Compiler Project

语法错误输出示例

```
tests/error_test_bison/missing_then_syntax.pas
```

```
*** ERRORS: PARSER (3) ***
```

```
[Parser][Line 6, Col 5] Syntax error at 'write':  
  expected {'}', ',', ';', ']', 'addop', 'do', 'downto',  
            'else', 'end', 'mulop', 'of', 'relop', 'then', 'to'} for 'term'  
[Parser][Line 6, Col 12] Expected 'then', got ')'  
[Parser][Line 6, Col 12] Syntax error at ')':  
  expected {';', 'begin', 'break', 'case', 'continue', 'else', 'end',  
            'for', 'id', 'if', 'read', 'while', 'write'} for 'statement'
```

05 SEMANTIC

语义分析

实现方法、支持类型、类型检查范围

Pascal-S 编译器验收展示

语义分析实现方法

Visitor 遍历 AST

SemanticAnalyzer : ASTVisitor

- analyze(ast) 清空错误并访问根节点
- 声明节点负责插入符号表
- 表达式节点负责类型推断和操作数检查
- 语句节点负责上下文规则检查

状态信息

- symTable_: 作用域栈
- currentFunction_: 当前函数名
- functionAssignedStack_: 函数是否给返回值赋值
- loopDepth_: break / continue 上下文
- caseDepth_: case 中允许 break

支持的类型系统

简单类型

integer、real、boolean、
char、string

数组类型

array[low..high] of
basic_type, 支持多维范围编码。

记录类型

record ... end 编码为
record{field:type;...}。

类型推断

- 数字带小数点推断为 real
- 根据操作数提升到 real
- / 结果为 real
- div / mod 结果为 integer

Pascal-S 编译器验证展示

赋值兼容

- 同类型可赋值
- integer 可提升到 real
- 数组整体不可赋值
- 其他跨类型赋值报错

2026 · Compiler Project

语义检查范围

声明与作用域

- 重复常量 / 变量 / 参数 / 子程序声明
- 未声明标识符、未声明函数、未声明过程
- 函数返回标识符与参数冲突
- 函数缺少返回值赋值
- 常量被赋值

表达式与运算

- 算术运算要求数值类型
- `div / mod` 要求整数
- `and / or` 要求布尔
- 一元 `+` / `-` 要求数值
- 关系运算要求两侧兼容

数组与 `record`

Pascal-S 编译器验收展示

- 数组下标必须为整数

参数、调用、语句

- 参数个数匹配

语义错误输出示例

参数个数错误

```
*** PARSE SUCCESSFUL ***

*** ERRORS: SEMANTIC (1) ***
  [Semantic][Line 9, Col 9]
    Function 'add' expects 2 argument(s), got
    1

Semantic analysis failed.
Code generation skipped.
```

为什么先输出 **Parse Successful**

语法正确不代表程序可编译；主流程先完成 LL(1) 语法分析并得到 AST，然后再进入语义阶段。

若语义阶段失败，主程序明确跳过代码生成，避免产生错误目标代码。

06 CODEGEN

代码生成

输入、采用技术、映射规则与算法思想

Pascal-S 编译器验收展示

代码生成输入与技术

输入

- 已通过语义分析的 AST
- 代码生成器内部维护一份符号表
- 类型信息来自声明节点和
`TypeInfo::fromString`
- 表达式生成时复用类型推断来选择格式符

采用技术

- `CCodeGenerator : ASTVisitor`
- 遍历 AST 节点并输出 C 代码
- `ostringstream out_` 累积结果
- `indent_` 管理缩进
- `helper` 函数处理类型映射、变量访问、表达式输出

代码生成算法思想

1. 注册声明

遍历 `const / var / subprogram` 时同步登记到生成器符号表。

2. 生成结构

全局声明和子程序先输出，再输出 `int main()`。

3. 递归表达式

表达式节点递归访问，运算符转换成 C 语义。

类型映射

- `integer` $\rightarrow$ `int`
- `real` $\rightarrow$ `float`
- `boolean` $\rightarrow$ `int`
- `char` $\rightarrow$ `char`

Pascal-S 编译器验收展示

运算与语句映射

- `and / or` $\rightarrow$ `&& / ||`
- `not boolean` $\rightarrow$ `!`
- `div / mod` $\rightarrow$ `/ / %`
- `for to` $\rightarrow$ `i <= end; i++`

2026 · Compiler Project

数组、参数与 I/O 生成

数组下界重定位

Pascal 数组可从任意下界开始; C 数组从 0 开始。

array[1..5] of integer

声明为长度 5; 访问时生成:

a[i - 1]

var 参数

Pascal 的引用参数在 C 中转指针:

- 形参: int *x
- 调用: &a
- 使用: (*x)

代码生成示例

Pascal-S

```
for i := 5 downto 1 do  
begin  
    write(i);  
end;
```

生成 C

```
for (i = 5; i >= 1; i--) {  
    printf("%d", i);  
}
```

record 字段访问

```
person.age = 20;  
write(person.age);
```

生成 C

```
person.age = 20;  
printf("%d", person.age);
```

错误处理与恢复策略

词法、语法、语义三阶段分层报告

Pascal-S 编译器验收展示

错误处理总览

词法错误

记录在 `Lexer::errors`; 扫描尽量继续, 最终输出 token 和错误列表。

语法错误

Parser 输出期望集合, 并利用 FIRST/FOLLOW 做 panic mode 恢复。

语义错误

AST 遍历阶段集中收集; 失败时停止代码生成。

统一格式

```
*** ERRORS: <PHASE> (N) ***
Pascal-S 编译器验收展示
[<Phase>][Line x, Col y] <message>
```

这样验收时可以清楚说明: 错误来自哪一阶段, 定位到哪里, 为什么错。

恢复策略细节

Lexer 层

- 遇到非法字符或非法字面量时记录错误
- 返回哨兵 token 后继续尝试扫描
- EOF 前统一检查未闭合括号
- 未闭合注释/字符串会保留起始位置

Parser 层

- 终结符不匹配时报告 `expected / got`
- 预测表无项时列出当前非终结符可接受 token
- 若当前 token 属于 FOLLOW，弹出非终结符
- 否则跳过输入直到 FIRST/FOLLOW 同步点

Semantic 层

Pascal-S 编译器验收展示

2026 · Compiler Project

语义阶段不做“猜测修复”，只收集所有可确定的错误；一旦存在错误，主流程输出 Code generation

08 DEMO

运行程序演示

测试用例的编译过程和结果

Pascal-S 编译器验收展示

演示用例安排

成功样例: 完整流水线

```
src/pascal_compiler.exe -i tests/  
correct_test/correct_all.pas -o /private/  
tmp/correct_all_demo.c
```

展示 parse、semantic、C codegen 全部通过。

词法错误样例

```
src/pascal_compiler.exe --lex tests/  
error_test_lex/invalid_number_3.pas
```

展示非法数字 1.2.3 的定位与错误信息。

语法错误样例

```
src/pascal_compiler.exe tests/  
error_test_bison/missing_then_syntax.pas
```

语义错误样例

```
src/pascal_compiler.exe tests/  
semantic_error_test/  
argument_count_mismatch.pas
```

Pascal9 编译器验收显示

2026 · Compiler Project

成功编译结果

运行输出

```
*** PARSE SUCCESSFUL ***
```

```
*** SEMANTIC ANALYSIS PASSED ***
```

```
*** C CODE GENERATED: /private/tmp/correct_all_demo.c ***
```

覆盖语法

Pascal-S 编译器验收展示
string、char、函数、case、
break、continue、for。

输出产物

指定 -o 后输出 C 源文件; --all
模式还会输出 token、AST、分析

继续验证

2026 · Compiler Project
可用 gcc 编译生成的 .c, 验证运
行时结果。

测试覆盖与当前状态

correct_test

正确程序完整流水线成功

error_test_lex

非法字符、数字、注释、括号等

error_test_bison

缺少 then/do/end/dot 等语法错误

语义错误语料

覆盖重复声明、未声明函数/过程、参数个数/类型、数组下标、for/while/case、break/continue 上下文、record 字段访问等。

Pascal-S 编译器验收展示

集成测试

tests/test_integration.cpp 覆盖 lexer、parser、AST、semantic、symbol table、codegen 的统一行为。

2026 · Compiler Project

答辩前建议运行:

注: 若修复或调整语义规则, 请在本页替换为最新测试统计。

源码讲解索引

词法

- `Lexer::keywordMap()`
- `Lexer::skipWhitespaceAndComments()`
- `Lexer::scanNumber()`
- `Lexer::scanIdentifierOrKeyword()`
- `Lexer::scanCharOrStringLiteral()`
- `tokenTypeToTerminal()`

语法 / AST

- `buildPascalSGrammar()`
- `Grammar::transformToLL1()`
- `Grammar::computeFirstSets()`
- `Grammar::buildParseTable()`
- `Parser::parse()`
- `Parser::registerDefaultActions()`

语义 / 符号表

Pascal 9 编译诸验证展示

- `SymbolTable::declare()`

代码生成

- `CCodeGenerator::generate()`

2026 · Compiler Project

总结

完整流水线

从源程序读取、词法、语法、AST、语义检查到 C 代码生成，主流程可端到端运行。

可解释实现

通过 token、parse process、AST、错误阶段前缀和源码位置，便于验收现场讲清楚每一步。

可扩展结构

文法、AST、语义分析和代码生成都按模块拆分；新增语言结构时可以沿同一链路扩展。